PYTHON FOR ACTUARIES · NOTEBOOK 08

GLMs for Actuaries

A 90-Minute Practical Workshop

> **Disclaimer:** This notebook estimates the **average claim amount** (claim severity), not the premium. A premium also needs the **claim frequency**: how many claims a contract causes per year. That requires all contracts, including those without a claim, and how long each was insured. `vehicle_fraud.csv` contains only claims.

In this session, you will learn how a GLM estimates the average claim amount from several tariff features at once: as a base amount times one factor per feature.

- **Part 1**: From Group Means to Factors
- **Part 2**: The Gamma GLM
- **Part 3**: Predicting New Contracts

The explanations use small tables created directly in the code. All exercises use `vehicle_fraud.csv` from Notebook 03.

```
# Import required libraries
import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf

# Avoid truncation of DataFrame output in Jupyter
pd.set_option('display.expand_frame_repr', False)

# Display all columns in the DataFrame output
pd.set_option('display.max_columns', None)

# Define a separator for better readability
sep = "=" * 50
```

Part 1: From Group Means to Factors

A tariff describes the expected claim amount as a **base amount** times one **factor** per feature, e.g. one factor for the vehicle type and one for the coverage type. The factors are estimated from past claims.

1.1 Mean Claim Amount per Group

`claims` holds 12 claims of compact and luxury cars with comprehensive or liability-only cover.

```
# Past claims of two vehicle types and two coverage types
claims = pd.DataFrame({
    'vehicle_type': ['Compact', 'Luxury', 'Compact', 'Luxury', 'Compact', 'Luxury',
                     'Compact', 'Luxury', 'Compact', 'Luxury', 'Compact', 'Luxury'],
    'coverage_type': ['Liability', 'Comprehensive', 'Comprehensive', 'Comprehensive',
        'Liability', 'Liability',
                      'Liability', 'Comprehensive', 'Comprehensive', 'Liability',
                          'Liability', 'Comprehensive'],
    'claim_amount': [2500.0, 8000.0, 5400.0, 10000.0, 3500.0, 4000.0,
                     2800.0, 8600.0, 6600.0, 5000.0, 3200.0, 9400.0]
})

print(f"\n{sep}\nPast Claims: claims\n{sep}")
print(claims)
```

```
================================================
Past Claims: claims
================================================
   vehicle_type  coverage_type  claim_amount
0       Compact      Liability        2500.0
1        Luxury  Comprehensive        8000.0
2       Compact  Comprehensive        5400.0
3        Luxury  Comprehensive       10000.0
4       Compact      Liability        3500.0
5        Luxury      Liability        4000.0
6       Compact      Liability        2800.0
7        Luxury  Comprehensive        8600.0
8       Compact  Comprehensive        6600.0
9        Luxury      Liability        5000.0
10      Compact      Liability        3200.0
11       Luxury  Comprehensive        9400.0
```

`groupby()` computes the mean claim amount per vehicle type (Notebook 03).

```
# Mean claim amount per vehicle type
vehicle_means = claims.groupby('vehicle_type')['claim_amount'].mean()

print(f"\n{sep}\nMean Claim Amount by Vehicle Type: claims.groupby('vehicle_type')
    ['claim_amount'].mean()\n{sep}")
print(vehicle_means)
```

```
================================================
Mean Claim Amount by Vehicle Type: claims.groupby('vehicle_type')['claim_amount'].mean()
================================================
vehicle_type
Compact    4000.0
Luxury     7500.0
Name: claim_amount, dtype: float64
```

1.2 Factors Relative to a Base Class

A factor compares a class with a **base class**. Here, the base class is `Compact`, the first class in alphabetical order. Each mean is divided by the mean of the base class:

vehicle_type	Mean claim amount	Factor
Compact	4,000	4,000 / 4,000 = 1.00
Luxury	7,500	7,500 / 4,000 = 1.875

The base class always has the factor 1. Factors computed from one feature at a time are **raw factors**.

```
# Raw factors relative to the base class Compact
vehicle_factors = vehicle_means / vehicle_means['Compact']

print(f"\n{sep}\nMean Claim Amount: vehicle_means\n{sep}")
print(vehicle_means)

print(f"\n{sep}\nRaw Factors: vehicle_means / vehicle_means['Compact']\n{sep}")
print(vehicle_factors.round(3))
```

```
================================================
Mean Claim Amount: vehicle_means
================================================
vehicle_type
Compact    4000.0
Luxury     7500.0
Name: claim_amount, dtype: float64

================================================
Raw Factors: vehicle_means / vehicle_means['Compact']
================================================
vehicle_type
Compact    1.000
Luxury     1.875
Name: claim_amount, dtype: float64
```

The same for `coverage_type`. Its base class is `Comprehensive`.

```
# Raw factors relative to the base class Comprehensive
coverage_means = claims.groupby('coverage_type')['claim_amount'].mean()
coverage_factors = coverage_means / coverage_means['Comprehensive']

print(f"\n{sep}\nMean Claim Amount by Coverage Type: claims.groupby('coverage_type')
    ['claim_amount'].mean()\n{sep}")
print(coverage_means)

print(f"\n{sep}\nRaw Factors: coverage_means / coverage_means['Comprehensive']\n{sep}")
print(coverage_factors.round(3))
```

```
================================================
Mean Claim Amount by Coverage Type: claims.groupby('coverage_type')['claim_amount'].mean()
================================================
coverage_type
Comprehensive    8000.0
Liability        3500.0
Name: claim_amount, dtype: float64

================================================
Raw Factors: coverage_means / coverage_means['Comprehensive']
================================================
coverage_type
Comprehensive    1.000
Liability        0.438
Name: claim_amount, dtype: float64
```

According to the raw factors, a luxury car costs 1.875 times as much as a compact car, and liability-only cover costs 0.44 times as much as comprehensive cover.

1.3 Two Features at Once

Raw factors are misleading when the features are related. First, count the claims per combination (Notebook 03):

```
# Number of claims per vehicle type and coverage type
print(f"\n{sep}\nGroup Sizes: claims.groupby(['vehicle_type', 'coverage_type'])
    ['claim_amount'].count().unstack()\n{sep}")
print(claims.groupby(['vehicle_type', 'coverage_type'])['claim_amount'].count().unstack())
```

```
==================================================
Group Sizes: claims.groupby(['vehicle_type', 'coverage_type'])['claim_amount'].count().unstack()
==================================================
coverage_type  Comprehensive  Liability
vehicle_type
Compact                    2          4
Luxury                     4          2
```

Luxury cars are mostly fully insured, compact cars mostly liability-only. Now the mean claim amount per combination:

```
# Mean claim amount per vehicle type and coverage type
print(f"\n{sep}\nMean Claim Amount: claims.groupby(['vehicle_type', 'coverage_type'])
    ['claim_amount'].mean().unstack()\n{sep}")
print(claims.groupby(['vehicle_type', 'coverage_type'])['claim_amount'].mean().unstack())
```

```
==================================================
Mean Claim Amount: claims.groupby(['vehicle_type', 'coverage_type'])
['claim_amount'].mean().unstack()
==================================================
coverage_type  Comprehensive  Liability
vehicle_type
Compact               6000.0     3000.0
Luxury                9000.0     4500.0
```

With the same coverage type, a luxury car costs 1.5 times as much as a compact car:

coverage_type	Compact	Luxury	Luxury / Compact
Comprehensive	6,000	9,000	9,000 / 6,000 = 1.5
Liability	3,000	4,500	4,500 / 3,000 = 1.5

The raw factor 1.875 is too high: luxury cars are more often fully insured, so their mean also contains the higher claim amounts of comprehensive cover. In the same way, the raw factor 0.44 for liability-only cover is too low; with the same vehicle type, it is 0.5. A GLM estimates all factors at once and separates the two effects.

> **Exercise 1: Raw Factors on `vehicle_fraud.csv`**
>
> Compute raw factors for vehicle type and coverage type and check how the two features are related.

Fraudulent claims would distort the average claim amount of honest customers, so all exercises use only legitimate claims. The first three code cells prepare the data: they remove the duplicates, add the age groups from Notebook 03 and keep the legitimate claims with a known age.

```
# Load the dataset and remove the duplicates
df = pd.read_csv('vehicle_fraud.csv')
df = df.drop_duplicates()
```

```
# Age groups as in Notebook 03
df['age_group'] = pd.cut(df['policyholder_age'], bins=[0, 30, 40, 50, 100], right=False,
                         labels=['<30', '30-39', '40-49', '50+'])
```

```
# Legitimate claims with a known age (the placeholder 0 means unknown)
legit_claims = df[(df['fraud'] == 0) & (df['policyholder_age'] >= 18)]
```

Task 1: Compute the mean claim amount per `vehicle_type` and the raw factors relative to `Compact`. Which vehicle type has the highest raw factor?

```
# Task 1: Raw factors of vehicle_type
# Hint: means_by_type = legit_claims.groupby('vehicle_type')['claim_amount'].mean()
#       raw_type_factors = means_by_type / means_by_type['Compact']
```

Task 2: Compute the raw factors of `coverage_type` relative to `Comprehensive`.

```
# Task 2: Raw factors of coverage_type
# Hint: means_by_coverage = legit_claims.groupby('coverage_type')['claim_amount'].mean()
#       raw_coverage_factors = means_by_coverage / means_by_coverage['Comprehensive']
```

Task 3: Count the claims per `vehicle_type` and `coverage_type`. Which vehicle types are mostly fully insured?

```
# Task 3: Number of claims per vehicle type and coverage type
# Hint: legit_claims.groupby(['vehicle_type', 'coverage_type'])
    ['claim_amount'].count().unstack()
```

Task 4: Compute the mean claim amount per `vehicle_type` and `coverage_type`. With `Liability` cover, how many times as much does a luxury claim cost as a compact claim? Compare with the raw factor from Task 1.

```
# Task 4: Mean claim amount per vehicle type and coverage type
# Hint: type_coverage_means = legit_claims.groupby(['vehicle_type', 'coverage_type'])
    ['claim_amount'].mean().unstack()
#       type_coverage_means.loc['Luxury', 'Liability'] / type_coverage_means.loc['Compact',
    'Liability']
```

Part 2: The Gamma GLM

A GLM (generalized linear model) estimates all factors at once. Logistic regression from Notebook 07 is a GLM for 0/1 targets. For claim amounts, a GLM needs a different distribution and link.

2.1 Distribution and Link

Choice	For claim amounts	Reason
Distribution	Gamma	Claim amounts are positive and right-skewed (Notebook 04)
Link	Log	The factors multiply

With the log link, the logarithm of the expected claim amount is a sum of coefficients β:

$$\log(\text{expected claim amount}) = \beta_0 + \beta_{\text{Luxury}} + \beta_{\text{Liability}}$$

The exponential function turns the sum into a product:

$$\text{expected claim amount} = e^{\beta_0} \cdot e^{\beta_{\text{Luxury}}} \cdot e^{\beta_{\text{Liability}}}$$

e^{β_0} is the **base amount** of the base classes, every other e^{β} is a **factor**. A claim only gets the coefficients of its own classes: for a compact car, β_{Luxury} is left out.

2.2 Fitting with `smf.glm()`

`smf.glm()` describes the model with a **formula**: the target left of `~`, the features right of it, joined with `+`. Text columns become classes automatically, with the first class in alphabetical order as the base class. `family=` sets the distribution and the link. `fit()` estimates the coefficients.

```
# Gamma GLM with a log link
glm = smf.glm('claim_amount ~ vehicle_type + coverage_type', data=claims,
              family=sm.families.Gamma(link=sm.families.links.Log())).fit()
print('GLM fitted.')
```

```
GLM fitted.
```

2.3 Reading `summary()`

`summary()` shows the fitted model. The upper table repeats the setup: 12 claims, family `Gamma`, link `Log`. The lower table has one row per coefficient: `Intercept` is β_0, `vehicle_type[T.Luxury]` is the coefficient of `Luxury` compared with the base class `Compact`. The column `coef` holds the coefficients, the other columns show how precisely they are estimated.

```
# Summary of the fitted model
print(f"\n{sep}\nSummary: glm.summary()\n{sep}")
print(glm.summary())
```

```
================================================
Summary: glm.summary()
================================================
                 Generalized Linear Model Regression Results
==============================================================================
Dep. Variable:           claim_amount   No. Observations:                   12
Model:                            GLM   Df Residuals:                       9
Model Family:                   Gamma   Df Model:                           2
Link Function:                    Log   Scale:                         0.015309
Method:                          IRLS   Log-Likelihood:                -93.105
Date:                Fri, 30 Oct 2026   Deviance:                       0.13900
Time:                        15:51:07   Pearson chi2:                     0.138
No. Iterations:                     6   Pseudo R-squ. (CS):               1.000
Covariance Type:            nonrobust
================================================================================================
                                   coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------------------------
Intercept                        8.6995      0.071    121.783      0.000       8.560       8.840
vehicle_type[T.Luxury]           0.4055      0.076      5.351      0.000       0.257       0.554
coverage_type[T.Liability]      -0.6931      0.076     -9.148      0.000      -0.842      -0.545
================================================================================================
```

2.4 Base Amount and Factors

`glm.params` holds the coefficients. `np.exp()` turns them into the base amount and the factors.

```
# Base amount and factors
factors = np.exp(glm.params)

print(f"\n{sep}\nCoefficients: glm.params\n{sep}")
print(glm.params.round(4))

print(f"\n{sep}\nBase Amount and Factors: np.exp(glm.params)\n{sep}")
print(factors.round(3))
```

```
==================================================
Coefficients: glm.params
==================================================
Intercept                     8.6995
vehicle_type[T.Luxury]        0.4055
coverage_type[T.Liability]   -0.6931
dtype: float64

==================================================
Base Amount and Factors: np.exp(glm.params)
==================================================
Intercept                     6000.0
vehicle_type[T.Luxury]           1.5
coverage_type[T.Liability]       0.5
dtype: float64
```

A compact car with comprehensive cover has an expected claim amount of 6,000 €. A luxury car costs 1.5 times as much, liability-only cover half as much.

2.5 GLM Factors vs. Raw Factors

```
# Raw factors and GLM factors side by side
comparison = pd.DataFrame({
    'raw': [vehicle_factors['Luxury'], coverage_factors['Liability']],
    'glm': [factors['vehicle_type[T.Luxury]'], factors['coverage_type[T.Liability]']]
}, index=['Luxury', 'Liability'])

print(f"\n{sep}\nRaw Factors vs. GLM Factors: comparison\n{sep}")
print(comparison.round(3))
```

```
==================================================
Raw Factors vs. GLM Factors: comparison
==================================================
             raw  glm
Luxury     1.875  1.5
Liability  0.438  0.5
```

The GLM finds the factors 1.5 and 0.5 from Section 1.3. The raw factors mix the two effects; the GLM separates them because it estimates all factors at once.

> **Exercise 2: Gamma GLM on `vehicle_fraud.csv`**
>
> Fit a Gamma GLM with four tariff features, read its factors and compare them with the raw factors.
>
> Continue with `legit_claims` from Exercise 1.

Task 1: Fit a Gamma GLM `glm_model` with a log link: target `claim_amount`, features `vehicle_type`, `coverage_type`, `region` and `age_group`. Show its summary.

```
# Task 1: Gamma GLM with four features
# Hint: glm_model = smf.glm('claim_amount ~ vehicle_type + coverage_type + region +
    age_group', data=legit_claims,
#                           family=sm.families.Gamma(link=sm.families.links.Log())).fit()
#       print(glm_model.summary())
```

Task 2: Compute the base amount and the factors `glm_factors`. Which classes are the base classes? Which vehicle type, coverage type, region and age group has the highest expected claim amount?

```
# Task 2: Base amount and factors
# Hint: glm_factors = np.exp(glm_model.params)
```

Task 3: Compare the GLM factors of `Luxury`, `Sports car` and `Liability` with the raw factors from Exercise 1. Where do they differ most? Why?

```
# Task 3: Raw factors vs. GLM factors
# Hint: factor_comparison = pd.DataFrame({
#           'raw': [raw_type_factors['Luxury'], raw_type_factors['Sports car'],
    raw_coverage_factors['Liability']],
#           'glm': [glm_factors['vehicle_type[T.Luxury]'],
    glm_factors['vehicle_type[T.Sports car]'],
#                   glm_factors['coverage_type[T.Liability]']]
#       }, index=['Luxury', 'Sports car', 'Liability'])
```

Part 3: Predicting New Contracts

3.1 `predict()`

`predict()` takes a DataFrame with the feature columns of the formula and returns the expected claim amount per row. The text columns need no preparation: the formula turns them into classes as in `fit()`.

```
# Three new contracts
new_contracts = pd.DataFrame({
    'vehicle_type': ['Compact', 'Luxury', 'Luxury'],
    'coverage_type': ['Comprehensive', 'Comprehensive', 'Liability']
})

print(f"\n{sep}\nNew Contracts: new_contracts\n{sep}")
print(new_contracts)
```

```
==================================================
New Contracts: new_contracts
==================================================
  vehicle_type  coverage_type
0      Compact  Comprehensive
1       Luxury  Comprehensive
2       Luxury     Liability
```

```python
# Expected claim amount per contract
predictions = glm.predict(new_contracts)

print(f"\n{sep}\nExpected Claim Amounts: glm.predict(new_contracts)\n{sep}")
print(predictions.round(2))
```

```
================================================
Expected Claim Amounts: glm.predict(new_contracts)
================================================
0    6000.0
1    9000.0
2    4500.0
dtype: float64
```

3.2 Checking a Prediction by Hand

The expected claim amount is the base amount times the factors of the contract's classes. Contract 2 is a luxury car with liability-only cover:

Step	Value
Base amount (Compact, Comprehensive)	6,000
× factor Luxury	1.5
× factor Liability	0.5
= expected claim amount	4,500

```python
# Base amount times the factors of contract 2
print(f"\n{sep}\nBase Amount and Factors: factors\n{sep}")
print(factors.round(3))

print(f"\n{sep}\nContract 2 by Hand: factors['Intercept'] *
    factors['vehicle_type[T.Luxury]'] * factors['coverage_type[T.Liability]']\n{sep}")
print(round(factors['Intercept'] * factors['vehicle_type[T.Luxury]'] *
    factors['coverage_type[T.Liability]'], 2))
```

```
================================================
Base Amount and Factors: factors
================================================
Intercept                     6000.0
vehicle_type[T.Luxury]           1.5
coverage_type[T.Liability]       0.5
dtype: float64

================================================
Contract 2 by Hand: factors['Intercept'] * factors['vehicle_type[T.Luxury]'] *
factors['coverage_type[T.Liability]']
================================================
4500.0
```

Exercise 3: Predicting New Contracts

Predict the expected claim amount of three new contracts and check one prediction by hand.

Continue with `glm_model` and `glm_factors` from Exercise 2.

Task 1: Create `new_policies` with three contracts and predict their expected claim amounts.

Contract	vehicle_type	coverage_type	region	age_group
0	Compact	Liability	R10	50+
1	SUV	Partial coverage	R05	30-39
2	Luxury	Comprehensive	R01	<30

```
# Task 1: Three new contracts
# Hint: new_policies = pd.DataFrame({
#           'vehicle_type': ['Compact', 'SUV', 'Luxury'],
#           'coverage_type': ['Liability', 'Partial coverage', 'Comprehensive'],
#           'region': ['R10', 'R05', 'R01'],
#           'age_group': ['50+', '30-39', '<30']
#       })
#       predicted = glm_model.predict(new_policies)
```

Task 2: Check the prediction of contract 1 by hand: base amount times its four factors.

```
# Task 2: Contract 1 by hand
# Hint: glm_factors['Intercept'] * glm_factors['vehicle_type[T.SUV]'] *
    glm_factors['coverage_type[T.Partial coverage]']
#       * glm_factors['region[T.R05]'] * glm_factors['age_group[T.30-39]']
```

Task 3: Contract 0 switches from `Liability` to `Comprehensive`. Predict its new expected claim amount. By which factor does it change? Compare with the factor of `Liability`.

```
# Task 3: Contract 0 with comprehensive cover
# Hint: upgraded = pd.DataFrame({'vehicle_type': ['Compact'], 'coverage_type':
    ['Comprehensive'],
#                                'region': ['R10'], 'age_group': ['50+']})
#       new_amount = glm_model.predict(upgraded)[0]
#       new_amount / predicted[0], 1 / glm_factors['coverage_type[T.Liability]']
```

Summary

- **Factors:** group means relative to a base class; raw factors mix the effects of related features
- **Gamma GLM:** `smf.glm()` with a formula and `family=sm.families.Gamma(link=sm.families.links.Log())`, `summary()`, base amount and factors with `np.exp()` of the coefficients
- **Predicting:** `predict()` on new contracts; expected claim amount = base amount × factors

The expected claim amount is not yet a premium: a premium also needs the claim frequency.

Next: detecting fraudulent claims in Notebook 09.

- statsmodels: Generalized Linear Models